

ASSIGNMENT

Software Engineering Principles (15SE202)

NAME: ANKITA PAL

REG. NO: RA1711003010611

FACULTY: MS. G. ABIRAMI

BATCH : 1

DATE : 27-03-2020

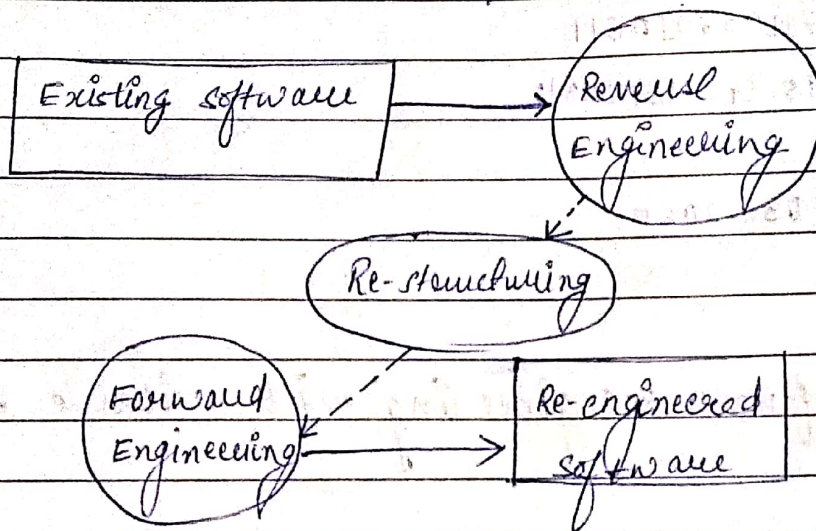
PAGES : 8

Q. Explain software Reengineering with suitable examples.
[15 marks].

A. Software Re-engineering

- (i) When we need to update the software to keep it to the current market, without impacting its functionality, it is called software re-engineering. It is a thorough process where the design of software is changed and programs are re-written.
- (ii) Legacy software cannot keep tuning with the latest technology available in the market. As the hardware becomes obsolete, updating of software becomes a headache. Even if software grows old with time, its functionality does not.
- (iii) For example, initially Unix was developed in assembly language. When C language came into existence, Unix was re-engineered in C, because working in assembly language was difficult.

- (iv) other than this, sometimes programmers notice that few parts of software need more maintenance than others and they need re-engineering.



Re-engineering Process

1. Decide what to re-engineer. Is it whole software or a part of it?
2. Perform reverse engineering, in order to obtain specification of existing software.
3. Restructured program if required. For example, changing function-oriented programs into object-oriented programs.
4. Re-structure data as required.
5. Apply forward engineering concepts in order to get re-engineered software.

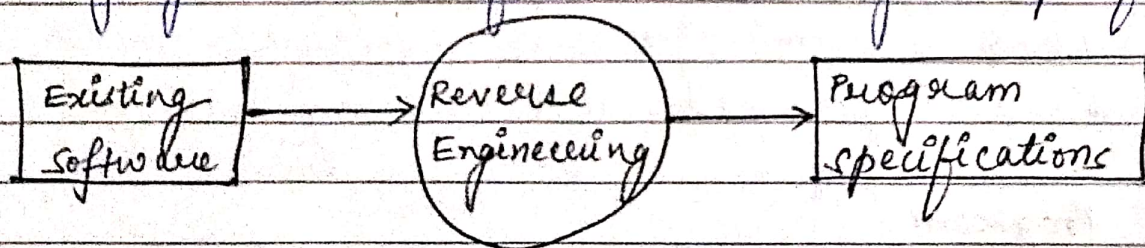
There are few important terms used in Software re-engineering.

Reverse Engineering

(i) It is a process to achieve system specification by thoroughly analysing, understanding the existing system. This process can be seen as reverse SDL model, i.e., we try to get higher abstraction level by analysing lower abstraction levels.

(ii) An existing system is previously implemented design, about which we know nothing. Designers then do reverse engineering by looking at the code and try to get the design. With design in hand, they try to conclude the specifications.

(iii) Thus, going in reverse from code to system specification.



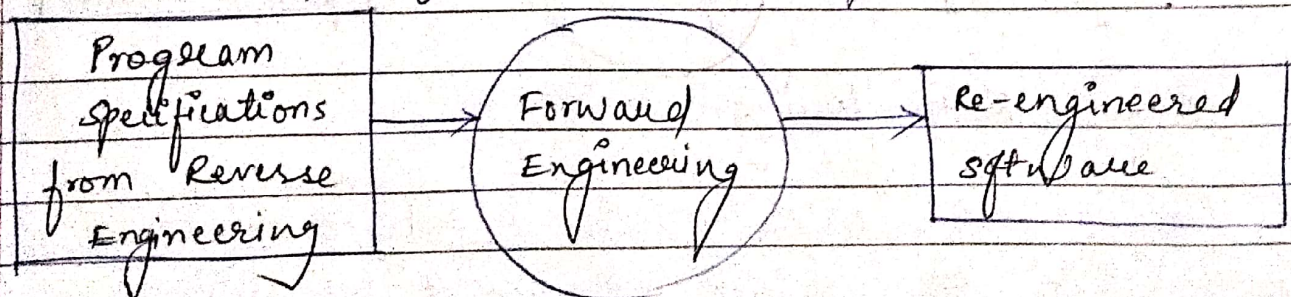
Program Restructuring

(i) It is a process to re-structure and re-construct the existing software. It is all about re-arranging the source code, either in same programming language or from one programming language to a different one. Restructuring can have either source code-constructing and data-restructuring or both.

- (ii) Re-structuring does not impact the functionality of the software but enhance reliability and maintainability. Program components, which cause errors very frequently can be changed, or updated with re-structuring.
- (iii) The dependability of software on obsolete hardware platform can be removed via re-constructing.

Forward Engineering

- (i) Forward Engineering is a process of obtaining desired software from the specifications in hand which were brought down by means of reverse engineering. It assumes that there was some software engineering already done in the past.
- (ii) Forward engineering is same as software engineering process with only one difference - it is carried out always after reverse engineering.

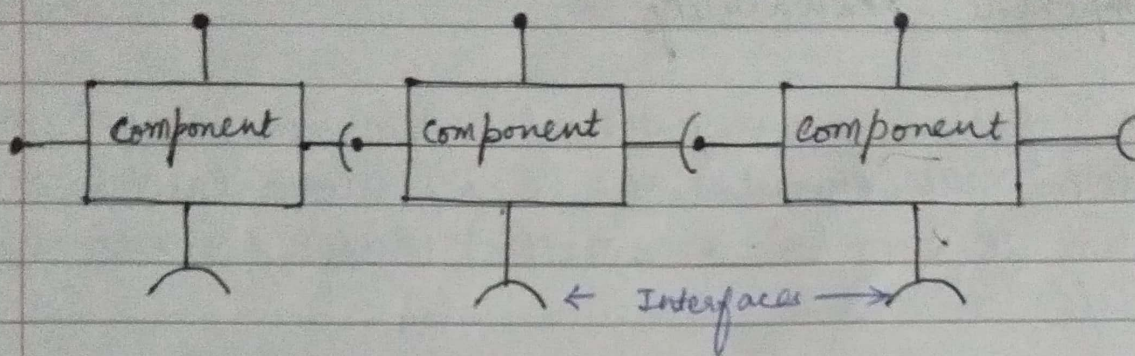


Component Reusability

A component is a part of software program code, which executes an independent task in the system. It can be a small module or sub-system itself.

Example

- (i) The login procedures used on the web can be considered as components, printing system in software can be seen as a component of the software.
- (ii) Components have high cohesion of functionality and lower rate of coupling, i.e., they work independently and can perform tasks without depending on other modules.
- (iii) In OOP, the objects are designed are very specific to their concern and have fewer chances to be used in some other software.
- (iv) In modular programming, the modules are coded to perform specific tasks which can be used across number of other software programs.
- (v) There is a whole new vertical, which is based on re-use of software component, and is known as component based software Engineering CBSE.



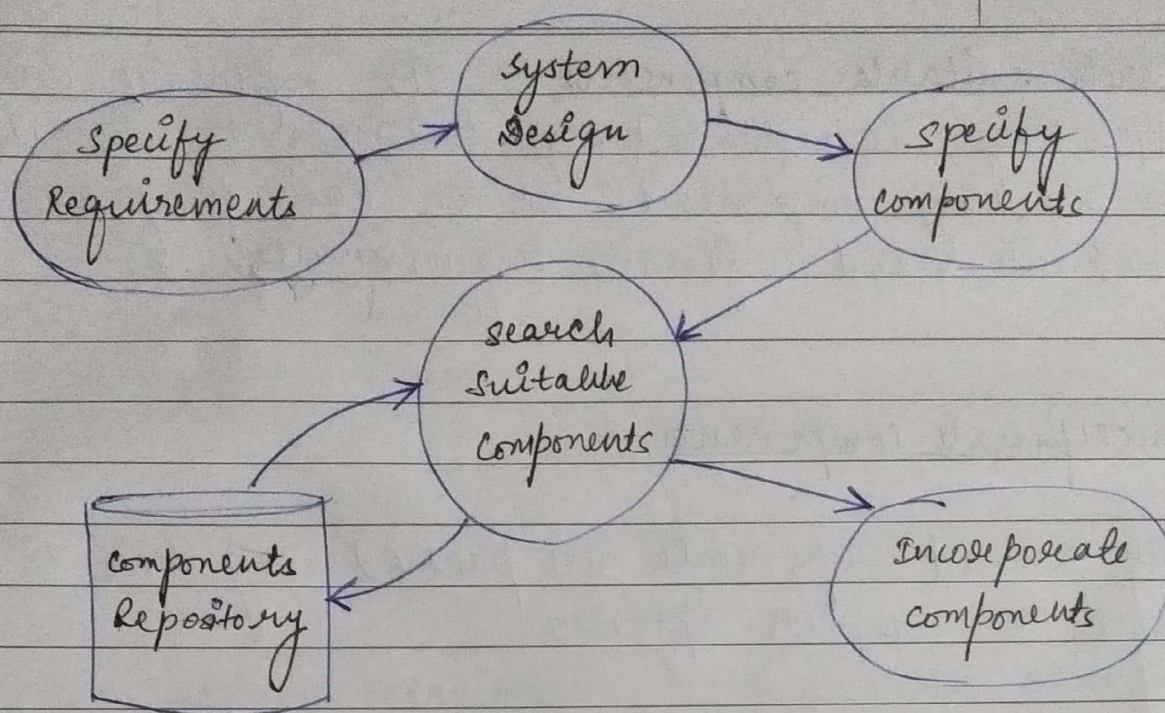
Re-use can be done at various levels :

1. Application level - Where an entire application is used as sub-system of new software.
2. Component level - where sub-system of an application is used.
3. Modules level - where functional modules are re-used.

software components provide interfaces, which can be used to establish communication among different components.

Re-use Process -

Two kinds of method can be adopted : either by keeping requirements same and adjusting components or by keeping components same and modifying requirements.



Requirement Specification -

The functional and non-functional requirements are specified, which a software product must comply to, with the help of existing system, user input or both.

Design -

This is also a standard SDLC process step, where requirements are defined in terms of software parlance. Basic architecture of system as a whole and its sub-systems are created.

Specify components - By studying the software design, the designers segregate the entire system into smaller components or sub-systems. One complete software design turns into a collection of a huge set of components working together.

Search suitable components - The software component repository is referred by designers to search for the matching component, on the basis of functionality and intended software requirements.

Incorporate components -

All matched components are packed together to shape them as complete software.